

Nelson Mbigili
Software Analytics
Handson Activity : Testing

Link to repository: <https://github.com/Nelsonmbigili/SAhandons-testing-nelson>

Task 1.1: Setup JaCoCo and Run the Coverage Report

Done

Task 1.2: Interpret the Coverage Report

1. What is the overall line coverage percentage for BankAccount?

Overall line coverage = (Covered lines / Total lines) * 100

Covered lines = Total lines - missed lines

Covered lines = 44 - 22 = 22

Overall line coverage = (22 / 44) * 100 = 50%

Overall line coverage of BankAccount is **50%**

2. What is the overall branch coverage percentage?

Overall branch coverage = (Covered branches / Total branches) * 100

Covered branches = Total branches - missed branches

Covered branches = 24 - 14 = 10

Overall branches coverage = (10 / 24) * 100 = 41.667 %

Overall branch coverage of BankAccount is **41.667 %** (Also indicated in report as **41%**)

3. Which methods in BankAccount have low coverage (below 60%)? Why do you think they are undertested?

Methods with coverage below 60% are :

- a. unfreeze(), freeze(), transfer(..), applyMonthlyInterest(..): 0% instruction Coverage.
- b. deposit(double): 58% Instruction Coverage.
- c. Constructor BankAccount(String, double): 58% Instruction Coverage.

4. Which methods have high coverage (above 90%)? Does high coverage guarantee the tests are effective? Explain your reasoning.

Methods with coverage above 90% are :

- a. getBalance(), getOwner(), isFrozen(): 100% instruction Coverage.

No, high coverage does not always guarantee effectiveness because coverage simply means the lines were executed during tests. But if the tests do not assert equality to expected results of the code, coverage may still be high even with incorrect code output from expected outputs.

5. Include a screenshot of your coverage report showing the overall percentages and the BankAccount class.

bank-demo > com.softwareanalytics.bank > BankAccount

BankAccount

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
getBalance()		100%		n/a	0	1	0	1	0	1
getOwner()		100%		n/a	0	1	0	1	0	1
isFrozen()		100%		n/a	0	1	0	1	0	1
withdraw(double)		85%		83%	1	4	1	8	0	1
BankAccount(String, double)		58%		50%	3	4	3	11	0	1
deposit(double)		58%		50%	2	3	2	6	0	1
applyMonthlyInterest(double)		0%		0%	3	3	5	5	1	1
transfer(BankAccount, double)		0%		0%	3	3	7	7	1	1
freeze()		0%		n/a	1	1	2	2	1	1
unfreeze()		0%		n/a	1	1	2	2	1	1
Total	88 of 161	45%	14 of 24	41%	14	22	22	44	4	10

Task 1.3 Run Mutation Testing and Identify a Surviving Mutant

Running: mvn test org.pitest:pitest-maven:mutationCoverage

1. What is the mutation score (percentage of mutants killed)? How many mutants generated and survived?

The mutation score is: **47%**

Mutants Generated: **30**

Mutants Killed: **14**

Mutants Survived: 30 - 14 = **16**

2. Include a screenshot of your mutation testing report.

Pit Test Coverage Report

Package Summary

com.softwareanalytics.bank

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	50%	47%	78%

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BankAccount.java	50%	47%	78%

3. The mutant information — file, line, original vs. mutated code

Mutant information:

File: BankAccount.java

Line: 19

Original Code: `if (initialBalance < 0)`

Mutated Code: changed conditional boundary → `if (initialBalance <= 0)`

status : survived

4. A plain-English explanation of why no existing test caught this mutant — what scenario or input would expose the fault?

This mutant changed the rule for creating a new account. Originally, the code allowed starting an account with a zero balance or higher. However, the mutated code altered the logic so that the code now blocks starting with both zero balance and negative balance, requiring only positive balance. Because the available tests only use accounts with positive balances, the change was not noticed. Since no test specifically checked if a zero-dollar account was still allowed, the bug in mutated code was undetected and the mutant survived.

5. Was the mutated line covered by the tests (check JaCoCo)? If so, what does this tell you about coverage vs. mutation testing?

Line 19 was marked as covered by JaCoCo, meaning the line was successfully executed during the test run. However, PIT revealed that these tests failed to identify a logic bug at the code's boundaries. This demonstrates that line coverage does not guarantee test effectiveness. A test suite can achieve high coverage while still failing to validate critical boundary conditions, as seen in this case. Mutation testing helps to evaluate the quality and diagnostic of errors of available tests suite.

Task 2.1: Create the full-coverage Branch

Done

Task 2.2: Write Tests to Increase Coverage

Done

Task 2.3: Rerun Coverage and Mutation Testing

1. The new line coverage and branch coverage percentages
New line coverage is : **100%**
New branch coverage is: **100%**
2. The new mutation score and mutant counts (generated, killed, survived)
Mutation score is now: **83%**
Generated mutants: **30**
Killed mutants: **25**
Survived mutants: **5**
3. Include screenshots of both the updated JaCoCo report and the PIT report

Screenshot: JaCoCo report (with added tests)

bank-demo > com.softwareanalytics.bank > BankAccount										
BankAccount										
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
BankAccount(String, double)		100%		100%	0	4	0	11	0	1
withdraw(double)		100%		100%	0	4	0	8	0	1
applyMonthlyInterest(double)		100%		100%	0	3	0	5	0	1
deposit(double)		100%		100%	0	3	0	6	0	1
transfer(BankAccount, double)		100%		100%	0	3	0	7	0	1
freeze()		100%		n/a	0	1	0	2	0	1
unfreeze()		100%		n/a	0	1	0	2	0	1
getBalance()		100%		n/a	0	1	0	1	0	1
getOwner()		100%		n/a	0	1	0	1	0	1
isFrozen()		100%		n/a	0	1	0	1	0	1
Total	0 of 161	100%	0 of 24	100%	0	22	0	44	0	10

Screenshot: PIT report (with added tests)

Pit Test Coverage Report

Package Summary

com.softwareanalytics.bank

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	100%	83%	83%

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BankAccount.java	100%	83%	83%

4. Briefly explain: How did your new tests affect coverage? How did they affect the mutation score?

The new tests had a great impact in line coverage as it improved it from 50% to 100%. The added missing test cases make the current test suite execute all lines of code in the BankAccount.

The mutation score also improved from 47% to 83% this proves that the added tests were also effective in detecting flaws and bugs in the code rather than just executing it.

However, it is still the case that the mutation score is still not 100% even with line and branch coverage score being 100%, showing that the test suite can be further improved for logic assertions and boundary conditions.

Task 3.1: Run Mutation Testing on the reference-suite Branch

1. The mutation score on the reference-suite branch

The mutation score from the reference suite is **70%**

2. How many mutants generated, killed, and survived?

Generated mutants: **30**

Killed mutants: **21**

Survived mutants: **9**

3. Include a screenshot of the PIT report for the reference-suite branch

Screenshot: JaCoCo report (reference-suite branch)

bank-demo > com.softwareanalytics.bank > BankAccount

BankAccount

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
getOwner()		0%		n/a	1	1	1	1	1	1
isFrozen()		0%		n/a	1	1	1	1	1	1
BankAccount(String, double)		100%		100%	0	4	0	11	0	1
withdraw(double)		100%		100%	0	4	0	8	0	1
applyMonthlyInterest(double)		100%		100%	0	3	0	5	0	1
deposit(double)		100%		100%	0	3	0	6	0	1
transfer(BankAccount, double)		100%		100%	0	3	0	7	0	1
freeze()		100%		n/a	0	1	0	2	0	1
unfreeze()		100%		n/a	0	1	0	2	0	1
getBalance()		100%		n/a	0	1	0	1	0	1
Total	6 of 161	96%	0 of 24	100%	2	22	2	44	2	10

Screenshot: PIT report (reference-suite branch)

Pit Test Coverage Report

Package Summary

com.softwareanalytics.bank

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	95% 42/44	70% 21/30	78% 21/27

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
BankAccount.java	95% 42/44	70% 21/30	78% 21/27

Report generated by [PIT](#) 1.15.5

Task 3.2: Compare the Two Suites

Comparison table or summary

Metric	my full-coverage branch	reference-suite branch
Line coverage	100%	95.45% *
Branch coverage	100%	100%
Mutation score	83%	70%
Mutants killed	25	21
Mutants survived	5	9

* line coverage of reference-suite branch

Overall line coverage = (Covered lines / Total lines)*100

Covered lines = Total lines - missed lines

Covered lines = 44 - 2=42

Overall line coverage = (42 / 44)*100 = **95.45%**

Overall line coverage of BankAccount is **95.45%**

Instruction coverage reported as **96%**

Answer the following in your report:

1. Both suites achieve ~100% coverage. Why do they have different mutation scores?

This happens because mutation scores evaluate the quality of the test suit while coverage evaluates lines executed during testing. So the difference happens because while all lines run during tests, that does not mean the tests explore all possibilities for errors like boundary conditions and code logic. Thus the test suit can be improved!.

2. What does this comparison tell you about the relationship between coverage and test effectiveness?

This comparison highlights that coverage tests are necessary in finding dead code in the code, because it makes sure that all available code is run. However to assert effectiveness that is to make sure the code performs what it is intended to, the mutation score is a better indicator. Therefore higher mutation score means it is likely that the test suite will catch a bug but higher line coverage means all the lines were executed.

3. In your own words: is mutation score a better indicator of test quality than coverage alone? Do you think common mutation techniques (e.g., switching operators etc) are effective at simulating real faults to determine test effectiveness?

Yes, I believe mutation score is a better indicator of test quality than coverage. However I think coverage is also important as it helps to actually evaluate how useful lines of code are.

Yes, I think mutation techniques are effective in simulating real possible bugs that may occur. Considering errors like boundary errors, it is so easy to get them wrong. Also the robustness of the possibilities created help with catching bugs that a developer may not have expected.

Acknowledgement and references

I acknowledge the use of [Gemini \(Google's AI\)](#) as a collaborative point of reference for getting familiar and understanding the tool documentation for PIT, JaCoCo and setting up the environment for and running java programs.